{ medtrics }

PRACTICE MEDICINE, NOT SOFTWARE

INTERNAL — ENGINEERING

medtrics-release-notes

> Engineering handoff, refreshed for **v0.20.0**: the architecture as built, the single combined Slack post, the Mem-only release archive, and a reconciliation of what changed since the 0.15.0 audit.

PLUGIN VERSION	0.20.0
HANDOFF DATE	3 September 2026
SCOPE	5 commands · 9 skills · 12 scripts · 313 passing tests
SUPERSEDES	The v0.15.0 forensic audit (21 August 2026)
PREPARED FOR	The next developers on this system

Internal engineering document. This is a **refreshed handoff** grounded in the current repository — README, CHANGELOG, the command and skill files, and the script inventory. It is **not** a re-run of the 0.15.0 forensic audit: the earlier defect register is reconciled at the structural level (§5), and claims tied to files that no longer exist are marked moot, not re-verified line by line. Items still living in surviving scripts are flagged as carried-forward and not re-audited.

Contents

00 Read this first

01 Architecture as built (v0.20.0)

02 What lands in Slack now

03 The Mem layer — directory, overrides, archive

04 The release archive, in depth

05 What changed since the 0.15.0 audit

06 Open items & recommended next

A Appendix — file & command map

0. Read this first

The 0.15.0 audit opened by saying the customer-facing pipeline had never posted, the internal digest was unreadable, and there was no test suite, config layer, audit trail, or cost visibility. Three things are different now, and they reframe the whole document.

1 **There is one combined post now, and it does post.**

The former split — a customer-facing `/release-slack` plus an internal `/internal-notes` — was merged into a **single internal message**. Each ticket card carries a client-portable `For clients:` lede next to the internal detail (ticket IDs, developer @-mentions, client names, Optimus URLs). The team ports the lede to clients. `/oracle-release` is the same shape for the Oracle product. Both preview in chat and never post without an explicit `y`.

2 **Whole subsystems the audit flagged are gone.**

The `/client-brief` and `/covista-weekly` commands, the QA-GIF path, and the ReportLab client-PDF renderer were removed. The plugin no longer generates customer PDFs at all. Every audit defect anchored to those files (§5) is moot — the code it cites is deleted, not patched in place.

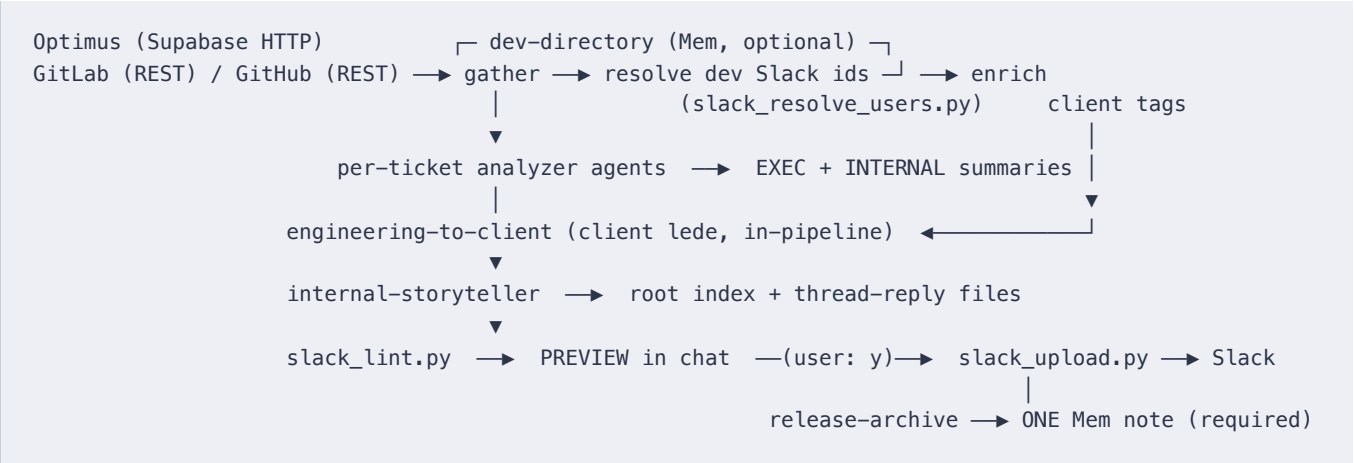
3 **There is a test suite, a Mem persistence layer, and deterministic scripts around every side effect.**

A `tests/` directory now ships out-of-bundle with **313 passing tests**. A Mem layer backs three concerns — a developer directory, Oracle PR overrides, and the release archive. Rendering, parsing, and matching are done by pure Python scripts; the commands only orchestrate.

> **What this refresh does not claim**
>
> It does not re-verify the 0.15.0 defects that live in surviving scripts (chiefly `gather_notes_data_optimus.py` — pagination, timeouts, retry, error-as-empty). Those are carried forward in §5 and listed as re-audit candidates in §6. Treat any such item as *unverified in 0.20.0*, not as fixed.

1. Architecture as built (v0.20.0)

1.1 The shape



Oracle differs only in source (GitHub PRs, with a PR-discovery + Mem-override step) and one safety net — the client ledes pass `ip_guard.py` before posting, because Oracle is pre-GA.

1.2 Command inventory

COMMAND	REAL OUTPUT	GATE	STATUS
`/release-slack`	One combined post to `#dream-team` (root index + thread cards)	preview → `y`	**Working; Mem-archived**
`/oracle-release`	One combined post to `#pod-oracle` (same shape; GitHub PRs)	preview → `y`	**Working; Mem-archived**
`/notes`	Deploy + integration markdown, in parallel	none	**Working**
`/deploy-notes`	`deploy/release-notes-<date>.md`	none	**Working**
`/integration-notes`	`integration/integration-notes-*.md`	none	**Working**

Retired since 0.15.0: `/internal-notes` (folded into the combined post), `/client-brief` and `/client-copy` (the `engineering-to-client` skill absorbed both), and `/covista-weekly`.

1.3 Skill inventory (9)

SKILL	ROLE
`internal-storyteller`	Composes the combined post — root index + per-ticket thread cards — for both release commands.
`engineering-to-client`	Writes each card's client-facing title and `For clients:` paragraph, in-pipeline, after the analyzers.
`release-archive`	The Mem-only release archive (§4). Required; deterministic renderer is `release_archive.py`.
`dev-directory`	Optional Mem cache of developer Slack ids, keyed by email / login / name.
`pr-overrides`	Optional Mem store of human PR picks for Oracle tickets.
`ip-guard`	Confidentiality net over the Oracle client ledes before posting.
`storyteller · copy-audit`	Back the markdown-document commands (`/deploy-notes` , `/notes`).
`oracle-storyteller`	Parked (unused since 0.12.0); kept in case the separated Oracle design returns.

1.4 External surfaces

- **Optimus** — ticket source for both tracks (classic `M1-` / `GEN-`, Oracle `ORC-`), via the gather scripts.
- **GitLab** — merge requests for the classic track. **GitHub** — pull requests for Oracle, fetched only inside `gather_notes_data_oracle.py`.
- **Slack** — the bot token drives `chat.postMessage` and `chat.getPermalink` through `slack_upload.py`. The bot must be a member of each target channel.
- **Mem (MCP)** — developer directory and Oracle PR overrides are optional; the **release archive is required** (§4).

2. What lands in Slack now

One top-level message per deploy — an index — with all detail in its thread. Never a second top-level message; overflow spills into further thread replies.

2.1 The message

```
ROOT (index)
  🚀 *Release Notes — <date>*
  *Enhancements & New Features*
  1. M1-986 · <client-facing headline>
  *Bug Fixes*
  2. M1-1039 · <client-facing headline>
  Clients to notify: KFSHRC · UCSF

THREAD CARD (one per ticket)
  *1. M1-986 · <client-facing headline>*
  *_For clients:_* <client-portable paragraph>
  • Ticket: M1-986        • Developer: <@dev> (Name)
  • Internal: <technical summary>
  • URL: <Optimus>    • MRs/PRs: <links>    • Clients: <codes> | TBC (platform-wide)
```

The ticket id joins its headline with a middle dot (`·` , since 0.18.1) so the id reads as a distinct element. The client lede is written by `engineering-to-client`, so it is safe for the team to lift and send; the surrounding card stays internal.

The confirmation gate

Both commands preview the full root and every reply in chat and post only on an explicit `y` — a scheduled invocation still stops at the preview. On a partial failure the command reports which replies landed and the root `ts`; a resume re-posts only the missing replies and never re-posts the root.

3. The Mem layer

Three independent concerns use Mem. Two are optional caches; one is the required archive. All Mem access goes through the MCP from the command, while a deterministic script renders and parses every note.

USE	COLLECTION	REQUIRED?	SCRIPT
Developer directory	`Release Notes Developer Directory`	Optional	`dev_directory.py`
Oracle PR overrides	`Oracle PR Overrides`	Optional	`pr_overrides.py`
Release archive	`Release Notes Archive`	**Required**	`release_archive.py`

Developer directory caches each developer's Slack id keyed by email, GitHub/GitLab login, and full name; Step 3 prefills from it, resolves the rest with `slack_resolve_users.py`, then writes newly resolved developers straight back. **PR overrides** record the GitHub PR a human chose when the automatic Oracle sweep was wrong or ambiguous — never a cache, only human picks; a preview reply of `ORC-232 → 415` records one and re-runs. Both degrade to a one-line notice when Mem is absent and never block a post.

> **The archive is the exception — it is required**
>
> Because Mem is the only place a posted release is kept (§4), both release commands probe for the Mem MCP in Step 1 and stop before gathering when it is not connected. The two caches stay best-effort; the archive does not.

4. The release archive, in depth

The most recent change (0.20.0). Every posted release is archived in exactly one place — Mem — and nothing is written to disk. The `.claude/outputs/notes/release-slack/` and `/oracle/` transcript copies that earlier versions wrote are gone.

4.1 Guarantees

- **Required & front-loaded.** Mem is checked in Step 1 (hard stop when absent) — `dry-run` included.
- **One note per release session** into `Release Notes Archive` (both tracks share it; the id prefix disambiguates), resolved by a user-set `.env` name and created if absent.
- **Trace header + full transcript.** The note carries date, channel, root `ts`, Slack permalink, the posted ticket list, and the **posted root and every reply verbatim** under a `## Posted message` heading.
- **Resume runs archive.** When a partial failure is finished with `--thread-ts`, that run writes the note.
- **A failed write never un-posts.** If Mem is unreachable after the post, the command prints the rendered note in chat for a manual paste — never to disk, never a re-post.

4.2 Note schema

```
Release: 2026-09-02 · oracle · #pod-oracle
**Date / Product / Channel / Channel id / Root ts / Permalink / Item count**
**Tickets:** ORC-232, ORC-234, ORC-240        ← load-bearing: a Mem search for one id lands here
**Clients:** KFSHRC, CVM-NCSU
- ORC-232 · <client title> · ENHANCEMENT · KFSHRC · <PR urls>
## Posted message → ### Root / ### Reply n  (each fenced verbatim)
```

Rendering, parsing, and the drop rule (skip `REFACTOR` and malformed blocks) are unit-tested in `tests/test_release_archive.py`. The permalink is the one thing fetched at archive time, via `slack_upload.py --get-permalink`; on failure the line is omitted, never constructed.

5. What changed since the 0.15.0 audit

5.1 Evolution, version by version

VERSION	CHANGE THAT MATTERS FOR THIS HANDOFF
`0.16.0`	The `engineering-to-client` skill is introduced (then as a standalone `/client-copy`).
`0.17.0`	Mem developer directory added; `engineering-to-client` moves into the pipeline; new card layout; `/client-copy` removed.
`0.18.0`	Oracle PR discovery v2 (every PR, a primary, a decision) + optional Mem PR overrides.
`0.18.1`	Ticket id joined to title with a middle dot on every title line.
`0.19.0`	Optional Mem release archive introduced (trace-only, alongside a disk copy).
`0.20.0`	Mem becomes the **only** archive — required, full transcript, resume-aware; disk copies removed.

5.2 Reconciling the old gap register

Status legend — **Moot**: the cited file/command was removed. **Addressed**: fixed structurally and visible in the repo. **Superseded**: the surface it lived on was retired. **Carried forward**: still in a surviving script; **not re-audited** in this refresh.

OLD ID	WHAT IT WAS	STATUS NOW
C1	Secrets/dotfiles bundled by `build_plugin.py`	**Addressed** — bundler excludes `.env`, `.env.*` (prefix) and dotfiles; positive "no-secret" assertion not re-audited
C2	PAT exfil in `fetch_mr_qa_gifs.py`	**Moot** — file & QA-GIF path removed
C3–C5	`/client-brief` flags, substring client match, fabricated classification (`client_scope.py`, `scope_incremental.py`)	**Moot** — command & both files removed
C6–C7	XML crash & font registration in `render_client_brief_pdf.py`	**Moot** — the client-PDF renderer is gone; the plugin no longer emits customer PDFs
C8	No grep for internal vocabulary in the client PDF	**Superseded** — client PDF retired; Oracle ledes now pass `ip_guard.py`, all posts pass `slack_lint.py`
H4	Breaking-change over-fire in `diff_signals.py`	**Moot** — file removed with the deterministic diff track
H8	GitLab failure read as empty in `integration_digest_data.py`	**Moot** — file removed; current integration path not re-audited
H9–H10	PARTIAL exit / non-atomic writes in `notes_tickets_gather.py`	**Moot** — file removed; write-atomicity across surviving scripts not re-audited
H1–H3, H5–H7	Pagination loop, no timeout, no retry, branch-name tickets, MR page cap, abandoned MRs — in `gather_notes_data_optimus.py`	**Carried forward** — file survives; **not re-audited** (see §6)

6. Open items & recommended next

1. **Run the full re-audit on the surviving gather path.** The H1–H3 / H5–H7 class of defects lived in `gather_notes_data_optimus.py`, which is still in service. Pagination termination, request timeouts, retry/backoff on Optimus errors, the 20-result MR cap, and branch-name-only ticket matching should be re-verified against the current file before they are trusted as fixed.
2. **Confirm the bundler's secret exclusion positively.** C1 is addressed by the exclude list; a post-build assertion that no `.env`/dotfile is ever present in the artifact would close it for good.
3. **Illustrative sample data.** The message examples in this document and in the operator runbook use placeholder tickets, developers, and client codes — they are not real posted data.
4. **The built `.plugin` is now tracked in git.** Only the current version is kept; delete the prior bundle on each release so "latest only" holds.

> **If you need the forensic depth**
>
> This refresh deliberately stops at structural reconciliation. For a new gap register with verified `file:line` claims across the current codebase — the equivalent of the 0.15.0 document — commission the full re-audit; it is a separate, larger pass.

Appendix A — File & command map

AREA	FILES
Commands	`release-slack · oracle-release · notes · deploy-notes · integration-notes`
Composition skills	`internal-storyteller · engineering-to-client · storyteller · copy-audit · ip-guard · oracle-storyteller (parked)`
Mem skills	`dev-directory · pr-overrides · release-archive`
Gather	`gather_notes_data_optimus.py · gather_notes_data_oracle.py · prepare_note_prompts.py`
Slack	`slack_resolve_users.py · slack_upload.py (+ --get-permalink) · slack_lint.py`
Mem (deterministic halves)	`dev_directory.py · pr_overrides.py · release_archive.py`
Safety / build	`ip_guard.py · build_plugin.py · assemble_oracle_root.py (parked)`
Tests	`tests/ — 12 modules, 313 tests (stdlib unittest; not bundled)`

Grounding sources for this handoff: `README.md`, `CHANGELOG.md` (0.15.0 → 0.20.0), the command and skill files, and the live `scripts/` and `tests/` inventories as of 3 September 2026.